

FraudLens

Agentic Fraud Investigation System

Full Project Scope · Adithya Raj · May 2026

Target Roles	Agentic AI · ML Engineer · MLOps
Dataset	PaySim — 6.3M transactions
Timeline	4 phases, no hard deadline
Score Impact	62/100 → 90+/100
Est. Monthly Cost	~\$0.29/mo (no AWS free tier)

What changed from v1

FraudLens v1 was a strong ML pipeline targeting Manulife. v2 is redesigned to make you a top candidate across three role types simultaneously: Agentic/GenAI Engineer, ML Engineer/Data Scientist, and MLOps/ML Platform Engineer.

The key architectural shift: the fixed pipeline becomes a LangGraph-orchestrated agentic system. The ML model, SHAP explainer, RAG retriever, and account velocity checker become tools the agent reasons over autonomously. A production React frontend is added. Nothing from v1 is removed — everything is elevated.

Infrastructure update: the project is designed to run at **~\$0.29/month on standard AWS on-demand pricing** (no free tier assumed). The HuggingFace Inference API is unaffected by AWS account age and remains free.

1. Strategic Overview

Why this architecture targets three role types

Most junior candidates build a fraud classifier and stop. The best add an explanation layer. FraudLens v2 goes further: it wraps those components as autonomous agent tools, adds a production UI, and ships with proper MLOps infrastructure.

Role Type	What They Look For	How FraudLens Delivers
Agentic / GenAI Engineer (e.g. RBC AiXel)	LangGraph, tool-use patterns, multi-step reasoning, MCP awareness	LangGraph orchestrator + 4 agent tools + structured output design
ML Engineer / Data Scientist (e.g. Manulife FWA)	Fraud detection, SHAP auditability, RAG pipelines, large-scale data eng.	XGBoost + SHAP + Dask 6.3M rows + FAISS RAG — all intact from v1
MLOps / ML Platform (any fintech)	Model serving, CI/CD, monitoring, containerised deployment	FastAPI + Docker + Lambda + CloudWatch drift monitoring + GitHub Actions

The single architectural shift

v1: data → model → SHAP → LangChain → RAG → report. Fixed sequence, no reasoning.

v2: transaction → LangGraph agent → agent decides which tools to call, in what order, based on what it sees → structured investigator report.

The ML model, SHAP explainer, regulatory retriever, and account velocity checker all still exist. They are now tools the agent invokes. Nothing is removed — everything gains a reasoning layer on top.

2. Full System Architecture

Layer overview

Layer	What It Does
Layer 1 — Data Engineering	Dask pipeline over 6.3M PaySim rows. Chunked ingestion, rolling velocity windows, balance anomaly detection, stratified sampling. Unchanged from v1.
Layer 2 — ML Core	XGBoost with SMOTE on TRANSFER + CASH-OUT types. Threshold-tuned precision-recall. Recall on fraud class is the headline metric.
Layer 3 — SHAP Explainability	TreeExplainer per-transaction attribution. Waterfall plots. Top contributing signals with directional impact. Feeds directly into agent tool output.
Layer 4 — LangGraph Agent	Orchestrates the full investigation. Decides which tools to call based on transaction context. Replaces the fixed LangChain pipeline from v1.
Layer 5 — Agent Tools	Four tools: run_fraud_model, explain_prediction, retrieve_regulations, check_account_velocity. Each wraps a FastAPI endpoint.
Layer 6 — RAG Pipeline	FAISS + sentence-transformers over OSFI/FINTRAC documents. Retrieves 2–3 regulatory passages per decision. Unchanged from v1.
Layer 7 — React Frontend	Production UI. Transaction input form, live agent reasoning trace (SSE), SHAP visualisation, investigator report display.
Layer 8 — FastAPI + MLOps	6 REST endpoints, Docker/Lambda, CloudWatch warm ping, drift monitoring, GitHub Actions CI/CD, pytest suite.

Agent tool definitions

Tool Name	When Agent Calls It
run_fraud_model(transaction)	Always called first. Returns risk score 0–1 and predicted class.
explain_prediction(transaction_id)	Called when risk score > 0.5. Returns SHAP attribution scores for top features.
retrieve_regulations(fraud_type)	Called when fraud type is identified. Queries FAISS index for OSFI/FINTRAC passages.
check_account_velocity(account_id)	Called when balance discrepancy is flagged. Returns rolling 1hr/3hr/24hr transaction counts.

3. React Frontend — The Demo Layer

Why the frontend matters

A live, interactive demo is the difference between a portfolio project and a product. When a hiring manager at RBC or Manulife clicks your GitHub link, they should be able to enter a transaction and watch the agent reason through an investigation in real time. That experience is not achievable with a raw API URL.

Page structure

Component	Content
Landing / Hero	Project title, one-line description, 'Try It' CTA, live Lambda status indicator.
Transaction Input Form	Fields matching PaySim schema: type (dropdown), amount, sender/receiver balances. Pre-filled fraud and legitimate examples.
Agent Reasoning Trace	Real-time SSE stream of agent steps: tool calls, scores, feature flags — the agent thinking, not just the output.
SHAP Visualisation	Recharts horizontal bar chart. Red = fraud-increasing features, blue = fraud-decreasing.
Investigator Report	Risk score badge (High/Medium/Low), plain-language explanation, regulatory passages, recommended action.
Architecture Tab	Static SVG of the full system stack for non-technical reviewers.

Technology choices

Component	Technology + Rationale
Framework	React 18 + Vite. Fast build, familiar to any frontend reviewer.
Styling	Tailwind CSS. Utility-first, keeps bundle small, professional without a design system.
Charts	Recharts for SHAP bar chart. React-native, no D3 complexity needed.
Agent trace streaming	Server-Sent Events (SSE) from FastAPI. Streams each agent step in real time. Simpler than WebSockets for one-directional streaming.
Hosting	AWS CloudFront + S3 static hosting. ~\$0.01/mo at demo traffic.

4. Four-Phase Build Plan

No hard deadline — complete phases sequentially. Phase 1 alone closes the fraud detection and data engineering gaps.

Phase 1 — ML Core (original week 1, unchanged)

Day	Task
Day 1	Download PaySim. Chunked ingestion (500k row chunks). Profile memory at 500k / 1M / 6.3M rows.
Day 2	Dask pipeline for full 6.3M rows. EDA: class distribution, transaction types, balance anomalies.
Day 3	Feature engineering: balance discrepancy, rolling velocity windows (1hr/3hr/24hr), log transforms.
Day 4	Filter to TRANSFER + CASH-OUT. SMOTE. Train XGBoost. Precision-recall curve. Tune threshold.
Day 5	SHAP TreeExplainer. Per-transaction attribution. Waterfall plots. Validate on 10 fraud samples.
Day 6	FastAPI skeleton: /predict and /explain endpoints. Test locally. Push to GitHub.
Day 7	Buffer. Write data engineering README section with memory numbers and pipeline notes.

Phase 2 — Learn Agents Properly (3–5 days, separate from FraudLens)

Do not skip this phase. Building agents without understanding the mental model produces code that runs but cannot be explained in interviews. LangGraph has a real learning curve.

- Complete the LangGraph official quickstart with two toy tools (calculator + mock search). Understand the graph, nodes, edges, and tool-calling loop.
- Read LangGraph 'Tool Use' and 'Multi-Agent' docs. Know the difference between an agent node and a tool node.
- Build one tiny agent unrelated to FraudLens — 50 lines, two mock tools. Goal is fluency, not functionality.
- Prepare answers: 'Walk me through how your agent decides which tools to call' and 'What happens if a tool call fails?'

Phase 3 — Agentic Layer + RAG + Full API

Day	Task
Day 8	Wrap Phase 1 FastAPI endpoints as LangChain tools using @tool decorator. Test each in isolation.
Day 9	Build LangGraph agent: graph, tool node, agent node, conditional edges. Test on 5 sample transactions.
Day 10	/velocity endpoint: rolling 1hr/3hr/24hr counts per account_id. Register as 4th agent tool.
Day 11–12	FAISS vector store. Chunk OSFI/FINTRAC PDFs. sentence-transformer embeddings. Build retriever.

Day	Task
Day 13	SSE streaming endpoint: streams each agent step as a Server-Sent Event.
Day 14	/full-report endpoint. 12+ pytest test cases across all endpoints.

Phase 4 — React Frontend + MLOps Hardening

Day	Task
Day 15–16	React + Vite + Tailwind scaffold. Transaction input form wired to /predict. Risk score badge.
Day 17	Agent reasoning trace: SSE consumer appending each step to a live log panel.
Day 18	Recharts SHAP bar chart from /explain response. Red/blue feature colouring.
Day 19	Investigator report component + architecture diagram tab (static SVG).
Day 20	AWS Lambda Docker deployment. CloudFront + S3 for React. EventBridge warm ping.
Day 21	/monitor endpoint: logs recall/precision to DynamoDB, returns drift flag when recall drops.
Day 22	GitHub Actions CI/CD. Full pytest suite. Final README with all required sections.

5. Infrastructure Cost Estimate

These figures assume a standard AWS account with no free tier active. All costs are at on-demand pricing for ~500 demo requests/month (typical portfolio traffic).

Monthly breakdown — no AWS free tier

Service	Cost & Rationale
AWS Lambda	~\$0.02/mo — \$0.20/1M requests + \$0.0000166667/GB-s. 500 req × 512MB × 3s = 768 GB-s. Negligible.
API Gateway	~\$0.01/mo — \$3.50/1M HTTP API calls. 500 demo calls/mo.
ECR (Docker image)	~\$0.25/mo — \$0.10/GB/mo. ~2.5GB image (HuggingFace client, not model weights).
CloudFront (React CDN)	~\$0.01/mo — \$0.0085/GB transfer. React bundle ~1MB, even 1,000 loads = 1GB.
S3 (frontend + FAISS)	~\$0.00/mo — ~50MB total. \$0.023/GB/mo = negligible.
DynamoDB (drift logs)	~\$0.00/mo — kilobytes of data at demo scale.
EventBridge (warm ping)	~\$0.00/mo — 4,320 pings/mo, \$1.00/1M events = \$0.004.
HuggingFace Inference API	\$0.00/mo — Not an AWS service. Free tier unaffected by AWS account age. ~150 reports/mo within limit.

Total estimated monthly cost

~\$0.29 / month

Expensive mistakes to avoid

Mistake	Cost + Why to Avoid
Provisioned Lambda concurrency	+\$40–100/mo. Completely unnecessary — EventBridge warm ping achieves the same demo reliability for \$0.
Loading model weights into Lambda container	+\$5–15/mo ECR storage + 90s cold starts. This is why the project uses HuggingFace Inference API instead of Ollama in Lambda.
Pinecone paid tier	+\$70+/mo. FAISS is free and fully sufficient for ~50 compliance documents at portfolio scale.
Forgetting to purge old ECR image layers	+\$2–10/mo over time. Delete untagged images monthly after rebuilds.

Free alternative (\$0/mo): Deploy FastAPI to Render.com free tier (750 hrs/mo, spins down after 15 min idle) + React on Vercel free tier. Tradeoff: 20–30s cold start after idle. Warm up the demo before any interview call. 'Deployed on AWS Lambda' reads better on a resume.

Set a AWS billing alert at \$5/month before deploying anything. It takes 2 minutes and catches misconfigurations before they compound.

6. Full Technology Stack

Category	Technologies
Data Engineering	Pandas (chunked), Dask 2024.5.0, NumPy, memory_profiler
ML / Modelling	XGBoost 2.0.3, Scikit-learn, imbalanced-learn (SMOTE)
Explainability	SHAP 0.44.0 (TreeExplainer), Matplotlib (waterfall plots)
Agentic AI	LangGraph (latest stable), LangChain 0.2.0, LangChain Tools
LLM Inference	HuggingFace Inference API (Mistral-7B-Instruct), Ollama (local dev only)
RAG	FAISS faiss-cpu 1.7.4, sentence-transformers, LangChain retriever
API	FastAPI, Pydantic, Uvicorn, SSE (Server-Sent Events)
Frontend	React 18, Vite, Tailwind CSS, Recharts, Axios
Cloud — Backend	AWS Lambda (Docker), API Gateway, S3, ECR, CloudFront
Cloud — Monitoring	CloudWatch EventBridge (warm ping), DynamoDB (drift logs)
DevOps	GitHub Actions (CI/CD), AWS SAM, pytest, Docker
Pinned Versions	langchain==0.2.0, xgboost==2.0.3, shap==0.44.0, faiss-cpu==1.7.4, dask==2024.5.0, huggingface-hub==0.23.0

Pin ALL versions in requirements.txt on Day 1. LangChain and LangGraph API changes between versions are the most common project blocker.

7. Resume Bullet Points

Write these before you build. Building exactly what these bullets describe keeps you on scope.

Project header line

FraudLens – Agentic Fraud Investigation System | LangGraph · LangChain · XGBoost · SHAP · RAG · FAISS · Dask · FastAPI · React · AWS

Bullets

- Designed a LangGraph-orchestrated fraud investigation agent that autonomously selects and sequences tools — fraud scoring, SHAP explanation, account velocity analysis, and regulatory retrieval — based on transaction risk context, replacing a fixed pipeline with dynamic multi-step reasoning.
- Engineered a large-scale fraud detection pipeline on the PaySim dataset (6.3M transactions) using Dask for memory-efficient chunked processing, partitioned rolling-window feature engineering, and stratified sampling — demonstrating production-grade data engineering at scale.
- Trained an XGBoost classifier with SMOTE oversampling on an extremely imbalanced dataset (~0.13% fraud rate), achieving [X]% recall on the fraud class via threshold-optimised precision-recall tradeoff analysis filtered to TRANSFER and CASH-OUT transaction types.
- Built a RAG pipeline using LangChain, FAISS, and HuggingFace sentence-transformers to retrieve relevant regulatory context from indexed OSFI/FINTRAC compliance documents, registered as an agent tool invoked dynamically during fraud investigations.
- Shipped a React frontend with real-time agent reasoning trace (Server-Sent Events), SHAP waterfall visualisations, and structured investigator report display — deployed to CloudFront with inference on AWS Lambda (Docker), CI/CD via GitHub Actions, and drift monitoring via CloudWatch + DynamoDB.

Fill in the [X]% recall metric once you have real results. Everything else is ready to paste.

8. What to Cut If Scope Gets Tight

Cut in this order — least impactful first. The bottom five are non-negotiable.

Component	Decision
GitHub Actions CI/CD	StockSenseAI already demonstrates CI/CD discipline. Safe to drop here.
DynamoDB drift monitoring	Add after applying. Doesn't affect ATS or initial screening.
/monitor endpoint	Mention as a 'planned extension' in the README if not built.
Architecture diagram tab in UI	Static SVG is a nice-to-have. README diagram is sufficient.
NEVER CUT: Dask pipeline	The 6.3M row data engineering story. Without it, the large dataset is meaningless.
NEVER CUT: LangGraph agent	The entire differentiator for agentic roles. Without it, v2 reverts to v1.
NEVER CUT: SHAP layer	Every candidate has a fraud classifier. The explanation layer is the differentiator.
NEVER CUT: RAG component	Named in both Manulife and RBC JDs. Guaranteed interview topic.
NEVER CUT: React frontend	Without a live demo, the project cannot be shown to non-technical stakeholders.

9. Interview Preparation

Prepare specific answers to these questions. They will be asked.

Agentic AI (RBC-specific)

Walk me through how your agent decides which tools to call.

The agent uses a ReAct-style loop in LangGraph. After each tool call, the agent node receives the tool output and decides whether to call another tool or terminate and produce the final report. If the fraud risk score is below 0.3, it skips SHAP explanation and regulatory retrieval entirely.

What happens if a tool call fails?

Each tool has a try/except wrapper that returns a structured error object. The agent handles failures gracefully — logs the failure, marks that component as unavailable in the report, and continues with remaining tools. Documented in the README as a design decision.

What is MCP and how does it relate to your agent tools?

Model Context Protocol standardises how AI agents interact with external tools and data sources. My agent tools follow the same tool-use pattern MCP standardises — each has a defined schema, input/output types, and is registered at initialisation. A production version would expose these tools via an MCP server rather than hardcoded tool registration.

Fraud and imbalanced data (Manulife-specific)

Why is accuracy a misleading metric here?

With 0.13% fraud rate, predicting all transactions as legitimate achieves 99.87% accuracy while catching zero fraud. Recall on the fraud class is the right primary metric — it measures the fraction of actual frauds caught.

How did you handle 6.3M rows without running out of memory?

Three stages: chunked pandas ingestion for EDA (500k row chunks, logged peak memory per chunk), Dask dataframe for feature engineering (processes full dataset without loading into RAM), stratified sampling to a model-training subset preserving the 0.13% fraud rate. Memory numbers documented at each stage in the README.

RAG and GenAI

What is RAG and why did you add it to a fraud classifier?

Retrieval-Augmented Generation: embed a document corpus, store in a vector index, retrieve semantically similar passages at inference time before generating. Added because fraud investigators need regulatory context alongside model decisions — a high-risk score means different things depending on OSFI reporting obligations. The RAG layer surfaces that context automatically.

Why FAISS over Pinecone or Weaviate?

FAISS runs locally with zero cost and zero latency overhead for a project-scale corpus (~50 documents). For production with millions of documents, multi-user access, and SLA requirements, Pinecone or Weaviate is more appropriate. This decision is explicitly documented in the README.

MLOps

How would you monitor this in production?

Track recall and precision on newly labelled fraud cases weekly. Monitor feature distribution drift using KS-test on amount and velocity features. The /monitor endpoint implements a simplified version — logs batch metrics to DynamoDB and flags when recall drops below a configurable threshold.

10. GitHub README Requirements

The README is the project's cover letter. A hiring manager will spend 60 seconds on it before deciding whether to read the code.

- Live demo link at the top — the CloudFront URL. Make it the first thing visible.
- One paragraph explaining the problem and why it matters in fintech and insurance.
- Architecture diagram: Raw CSV → Dask → XGBoost → SHAP → LangGraph Agent → [Tools] → RAG → FastAPI → React.
- Performance table: memory usage at each pipeline stage (chunk ingestion, Dask, feature engineering).
- Screenshot of the React UI showing a completed investigation with agent trace and SHAP chart.
- Sample investigator report — paste a real one from a high-confidence fraud transaction.
- Final metrics: recall, precision, F1 on fraud class (not accuracy).
- Setup instructions: clone, install requirements, set env vars, run locally.
- Architecture decision notes: why LangGraph over fixed pipeline, why Dask over Spark, why FAISS over Pinecone, why HuggingFace Inference API over Ollama in Lambda, why EventBridge over provisioned concurrency.

The architecture decision notes section separates a portfolio project from a homework assignment. Hiring managers want deliberate choices, not tutorial-following.

11. Architecture Decision Notes

Document each of these in your README. These are the answers interviewers probe for.

Decision	Your Reasoning
HuggingFace Inference API vs Ollama in Lambda	Ollama requires loading a 4–7GB model into the Lambda container, causing a 10GB+ image and 60–90s cold starts. HuggingFace Inference API offloads inference, keeps the Docker image under 3GB, and eliminates the cold start problem entirely.
EventBridge warm ping vs provisioned concurrency	Provisioned concurrency costs \$40–100/mo. An EventBridge scheduled rule every 10 min costs ~\$0.00/mo (4,320 pings vs 14M free events) and achieves the same demo reliability. Provisioned concurrency is right at production traffic; EventBridge is right for a portfolio demo.
FAISS vs Pinecone/Weaviate	FAISS runs locally with zero cost for ~50 compliance documents. Pinecone/Weaviate are more appropriate for production scale (millions of docs, multi-user, SLAs). Documented as a deliberate scope decision.
Dask vs Spark	Spark adds cluster orchestration complexity that is unnecessary for a single-machine 6.3M row dataset. Dask runs locally with a familiar pandas-like API and demonstrates the same distributed processing concepts with far less setup overhead.

Decision	Your Reasoning
XGBoost vs neural network	XGBoost produces SHAP-compatible feature attributions natively via TreeExplainer, which is exact and efficient. Neural network SHAP requires DeepExplainer or GradientExplainer, which are approximations and slower. Model auditability is a core requirement in regulated financial environments.
SSE vs WebSockets for agent trace	SSE is one-directional (server → client) which is exactly what agent step streaming requires. WebSockets add bidirectional complexity and connection management overhead that provides no benefit for this use case.

12. Updated Roadmap

Step	Action
Complete	StockSenseAI — full AWS deployment on resume.
Phase 1 (start here)	FraudLens ML core — Dask + XGBoost + SHAP + FastAPI skeleton.
Phase 2	LangGraph learning track — build toy agent before touching FraudLens.
Phase 3	Agentic layer — LangGraph orchestrator + 4 tools + RAG + SSE streaming.
Phase 4	React frontend + AWS deployment + MLOps hardening + README.
After Phase 4	Swap Audience Segmentation for FraudLens on resume. Maintain 3 projects.
Apply	RBC AiXel + Manulife FWA + any fintech ML/MLOps role. Projected match: 90+/100.

Generated from career coaching session — Claude, May 2026